TD 2 – Tarification automobile par Machine Learning

Mise en production d'une application de Machine Learning

Contexte

Dans ce TD, l'objectif va être de réaliser la mise en production des modèles que vous avez préalablement codés dans le cours de Data Science.

Comme nous l'avons vu pendant le cours, un modèle de Machine Learning / d'IA développé dans un notebook ne peut pas être mis à la disposition d'une entreprise pour une utilisation à grande échelle. Il y a un certain nombre d'étapes à respecter avant.

L'objectif concret de ce projet est de réaliser une API, qui permettra d'exposer / interfacer vos modèles.

Remarque : Sur l'ensemble du projet, l'ensemble des bonnes pratiques développées et présentées pendant le cours sont à mettre en place :

- Qualité du code
- Utilisation maîtrisée de Git et GitHub.

Notation

La note prendra en compte la qualité des développements, l'organisation du travail, la clarté des livrables ainsi que le respect des consignes, conformément aux critères suivants.

Aussi, l'évaluation portera sur la qualité de la restitution. Il sera évalué le fonctionnement global de l'application, la compréhension fine des travaux réalisés et la collaboration entre les membres du groupe (l'utilisation de l'IA est possible, néanmoins il est obligatoire de comprendre l'ensemble des réponses générées pour pouvoir répondre aux questions de l'évaluation et pas que... (Vidéo : Pourquoi comprendre ce que l'IA génère dans le cadre des études)).

1 Récupération des modèles et mise en commun

Durant le précédent TD, vous avez développé deux modèles du projet agrégés en un modèle commun (modèle de prédiction des montants + modèle de prédiction des fréquences –> modèle de prime).

Dans ce cadre, vous devez nettoyer et synthétiser vos travaux, sous format de script, pour pouvoir en faire une API. Vous devrez disposer des éléments suivants pour faire le développement :

- la partie liée aux traitements des données ;
- la partie liée à la prédiction des probabilités / fréquences / montants ;
- les notebooks de développement permettant de contrôler le développement et récupérer les informations connexes ;
- toute autre information ou développement jugé pertinent.

Ces documents sont à synthétiser sur un dépôt GitHub, qui servira d'espace central pour les rendus finaux.

Actions à réaliser

1. Récupérer les documents attendus ;
2. Centraliser ces documents sur un dépôt GitHub.

2 Traitements des données

Normalement, vous disposez de deux méthodologies de traitements des données, un pour chaque modèle. Ces traitements sont réalisés sur l'ensemble des données.

Ici, dans le cadre d'une API, le traitement des données se fera sur une observation. Cela implique des considérations importantes sur le développement et l'utilisation de ce traitement des données. Il sera probablement nécessaire d'en changer une partie.

Ensuite, les développements ont probablement été faits sans tests dédiés. Il sera nécessaire de s'assurer que le code développé est robuste aux travers de tests.

Actions à réaliser

1. Identifier les parties du traitement des données qui nécessitent un travail de formatage et les nettoyer ;
2. Intégrer ces traitements pour permettre le traitement d'un seul événement, au format JSON (ou assimilé) pour les deux modèles ;
3. Définir une stratégie de tests basée sur le cours et l'application à ce cas d'usage.

3 Utilisation des modèles

À ce stade, vous disposez d'un processing des données clair et compatible avec une utilisation unitaire (un seul cas à la fois et pas un dataframe avec des centaines de lignes).

Actions à réaliser

1. Vérifier que les modèles sont sauvegardés dans un format pertinent (pickle) ;
2. Tester avec un dictionnaire donné (JSON) une prédiction sur chacun des modèles.

4 Développement de l'API

Maintenant que l'on dispose d'un processing fonctionnel et de deux modèles propres, on peut passer à la partie industrialisation.

Dans ce contexte, il sera demandé de réaliser une API disposant de 4 routes :

- `/health` : vérifier l'état de santé de l'application ;
- `/predict_frequency` : fait la prédiction du modèle de fréquence ;
- `/predict_amount` : fait la prédiction du modèle de montant ;
- `/predict` : fait la prédiction agrégée.

Aussi, l'API nécessite d'autres éléments afin d'être parfaitement utilisable par une entreprise :

- Une documentation swagger / open API ;

- Des tests complets.

Contrat d'API

Afin de pouvoir tester votre API avec l'outil d'auto-évaluation fourni, celle-ci doit respecter le contrat suivant.

Route	Méthode	Exemple de réponse (JSON)
`/health`	GET	`{"status": "ok"}`
`/predict_frequency`	POST	`{"frequency": 0.12}`
`/predict_amount`	POST	`{"amount": 1500.0}`
`/predict`	POST	`{"frequency": 0.12, "amount": 1500.0, "premium": 180.0}`

Les trois routes de prédiction prennent en entrée un seul contrat, au format JSON, dont les champs sont les colonnes du jeu de données (hors variables cibles `nombre_sinistres` et `montant_sinistre`). Les champs `sex_conducteur2` et `anciennete_vehicule` peuvent valoir `null`. Exemple :

```
{
  "id_client": "A00000001", "id_vehicule": "V01", "id_contrat": "A00000001-V01",
  "bonus": 0.5, "type_contrat": "Maxi", "duree_contrat": 29,
  "anciennete_info": 9, "freq_paiement": "Biannual", "paiement": "No",
  "utilisation": "Retired", "code_postal": "36233", "conducteur2": "No",
  "age_conducteur1": 85, "age_conducteur2": 0,
  "sex_conducteur1": "M", "sex_conducteur2": null,
  "anciennete_permis1": 62, "anciennete_permis2": 0,
  "anciennete_vehicule": 10, "cylindre_vehicule": 1587, "din_vehicule": 98,
  "essence_vehicule": "Gasoline", "marque_vehicule": "PEUGEOT",
  "modele_vehicule": "306", "debut_vente_vehicule": 10,
  "fin_vente_vehicule": 9, "vitesse_vehicule": 182,
  "type_vehicule": "Tourism", "prix_vehicule": 20700, "poids_vehicule": 1210
}
```

Règles à respecter :

- les prédictions sont des nombres JSON (et non des chaînes de caractères), positifs ou nuls ;
- la prime est la prime pure : `premium` = `frequency` × `amount` ;
- une même entrée renvoie toujours la même prédiction ;
- une entrée invalide (champ manquant, mauvais type, corps vide) renvoie une erreur `4xx` (typiquement `422`), jamais une erreur `500` ;
- la documentation OpenAPI est accessible sur `/openapi.json` et `/docs`.

Actions à réaliser

1. Développer l'API dans un framework bien choisi ;
2. Ajouter les 4 routes attendues en respectant le contrat d'API ;
3. Ajouter le swagger de l'API ;
4. Définir une stratégie de tests adaptée et la mettre en pratique.

5 Intégration et développement

Cette partie comporte deux points majeurs :

- La réalisation d'une image Docker de l'API ;
- La réalisation d'une CI / CD dans GitHub avec tous les éléments précédents.

A ce stade, l'API est déployée en local et dispose de tests assurant sa fiabilité. L'objectif de cette partie est d'intégrer une CI / CD sur le projet.

On se propose donc de réaliser une image Docker incluant l'ensemble des parties prérequises.

Concernant la partie CI, on pourra proposer :

- une installation des dépendances avec le framework de versionnage des packages choisis ;
- une exécution automatique de tests ;
- une vérification automatique de la qualité du code.

Sur la partie CD, c'est plus compliqué car nous ne disposons pas d'un environnement dédié pour le déploiement des applications.

Actions à réaliser

1. Créer le fichier DockerFile et créer l'image Docker ;
2. Réaliser la CI du projet ;
3. Réaliser une activité de veille sur la partie CD et déploiement de l'image ;
4. Trouver une solution d'hébergement Open Source existe et déployer l'image.